

При кой тип граматики правилата изпълняват следното изискване: от лявата страна има единствен терминален символ, а от дясната страна има само един терминален символ или един терминален следван от един нетерминален символ или празен низ?

Граматики без ограничения (тип 0)

Контекстно-свободни граматики (тип 2)

Регулярни граматики (тип 3) ✓

Контекстно-зависими граматики (тип 1)

Дадена е граматиката  $G = \{ VT, VN, P, S \};$

$VT = \{ a, b \};$

$VN = \{ S, X, Y \};$

$S = (S);$

$P = \{ p_i \mid i = 1, 3 \};$

$P1: S \rightarrow aXa$

$P2: X \rightarrow \epsilon \mid bY$

$P3: Y \rightarrow \epsilon \mid cXc$

aba ✓

асса

abcbcbba

abcba

Коя от следните операции е достатъчна за преобразуване на произволна контекстно-свободна граматика в LL(1)?

Факторизация и премахване на лява рекурсия

Нито едно от изброените ✓

Премахване на лява рекурсия

Факторизация

Дадена е следната граматика  $S \rightarrow A111 \mid S1, A \rightarrow A0 \mid 00$ . Кой от следните езици поражда тази граматика?

$\{0n1m \mid n=1, m=5\}$

$\{0n1m \mid n=2, m=3\}$  ✓

$\{0n1m \mid n \text{ трябва да е по-голямо от две и } m \text{ трябва да е по-голямо от четири}\}$

Всички изброени

Нито един от изброените

При синтактичен анализ отдолу-нагоре разпознаването на изречението започва от:

(Изберете едно)

Стартов символ

Първи терминален символ

Низ от нетерминални символи

Низ от терминални символи ✓

Кое от следните твърдения е ГРЕШНО?

LR(k) синтактичен анализатор е по-моцнен от LL(1) анализатор

LR(k) граматика не може да бъде нееднозначна

При еднозначна граматика е възможно ляво и дясно извеждане на изречение ✓

LL(1) синтактичен анализатор изпълнява анализ отгоре-надолу

Изискване 2 за LL(1) е:

За правила от вида  $A: \epsilon \text{ first}(A) \mid \text{follow}(A) = \emptyset$

За правила от вида  $A: \epsilon \text{ first}(A) \cap \text{follow}(A) = U$  (универсално множество)

За правила от вида  $A: \epsilon \text{ first}(A) \mid \text{follow}(A) = U$  (универсално множество)

За правила от вида  $A: \epsilon \text{ first}(A) \cap \text{follow}(A) = \emptyset$  ✓

Изберете ГРЕШНОТО твърдение:

Синтактичното дърво може да бъде построено от корена или от листата

Нито едно от изброените ✓

Ако даден връх в синтактичното дърво не е листо, то той винаги е означен с нетерминал

В синтактичното дърво етикетите на листата са терминали

Граматиката  $S \rightarrow SS \mid 0S1 \mid 1S0 \mid \epsilon$  поражда низове, които съдържат:

Равен брой 0 и 1 ✓

Различен брой 0 и 1

Нито едно от изброените

Произволен брой 0, следвани от произволен брой 1

Дадена е следната граматика за пораждаване на изрази:

$E \rightarrow E * F \mid F + E \mid F$

$F \rightarrow F - F \mid id$

a) – е с по-висок приоритет от \* ✓

б) \* е с по-висок приоритет от +

в) + е с по-висок приоритет \*

г) + и – имат еднакъв приоритет

Какви лексеми се съдържат в следния текст на входна програма:

(Изберете едно)

```
while(i<=10) {j++;}
```

а) Една ключова дума, два идентификатора, пет разделители, два оператора, една константа ☒

б) Една ключова дума, три идентификатора, пет разделители, два оператора

в) Една ключова дума, два идентификатора, пет разделители, четири оператора, една константа

г) Една ключова дума, два идентификатора, пет разделители, три оператора, една константа

Интерпретаторът изпълнява следните действия:

а) Изпълнява директно операциите в програмата ☒

б) Генерира изходна програма

в) Създава изпълним файл

г) Намалнява броя на грешките във входната програма

Коя от следните граматики поражда регулярни множества?

а) Нито една от изброените

б) Всички изброени

в) Граматика на английски език

г) Контекстно-свободна граматика ☒

д) Само граматика, която не е контекстно-свободна

Кое твърдение е ГРЕШНО за LR(k) парсер

а) Изпълнява синтактичен анализ отдолу-нагоре

б) Поглежда напред в потока от лексеми

в) Изпълнява синтактичен анализ отгоре-надолу ☒

г) Чете поток от лексеми

6. Дадена е следната граматика G:

S -> F | H

F -> p | c

H -> d | c

Твърдение 1: Може да се построи LL(1) парсер за извеждане на всички низове, породени от G.

Твърдение 2: Може да се построи LR(1) парсер за извеждане на всички низове, породени от G.

а) Вярно е само твърдение 2

б) Нито едно от двете твърдения не е вярно ☐

в) Верни са и двете твърдения

Вярно е само твърдение 1

Как се нарича програма, която генерира изпълним формат от отделно компилирани модули на входна програма?

а) Асемблер

б) Свързващ редактор ☐

в) Компилятор

Интерпретатор

Генератор

Кой символ от граматиката означава прилагане на част от правило 0 или 1 път? (Свободен отговор)

? ☐

Коя от следните операции е достатъчна за преобразуване на произволна контекстно-свободна граматика в LL(1)

Нито едно от изброените ☐

Премахване на лява рекурсия

Факторизация

Факторизация и премахване на лява рекурсия

Коя операция не се изпълнява при синтактичен анализ отдолу-нагоре?

Проверка за съвместимост на типове ☐

Преместване в ново състояние

Редуциране

Откриване на грешки

Кой символ означава, че символ (или низ от символи) от граматиката се използва точно 1 път?

\*

?

|

+

(част от правило) ☐

Даден е регулярен израз  $R$  на езика  $L$  върху азбуката  $\{a, b\}$  и низ  $w$ :

$$R = (ba^*bb) + (ba^*ba + a(b + a + b^*a)), w = aabba$$

а)  $R$  не разпознава  $w$  ☐

б)  $R$  разпознава  $w$

в)  $R$  не е регулярен израз

г)  $w$  не е низ от езика  $L$

Дадена е граматиката  $G = \{ VT, VN, P, S \}$ ;

$$VN = \{ A, B, D \};$$

$$S = \{ D \};$$

$$P = \{ p_i \mid i=1,7 \};$$

$$P1: A \rightarrow (D);$$

$$P2: D \rightarrow D + A;$$

$$P3: D \rightarrow A;$$

$$P4: B \rightarrow a;$$

$$P5: B \rightarrow b;$$

$$P6: A \rightarrow A * B;$$

$$P7: A \rightarrow B ;$$

а) контекстно-зависима

б) неправилна

в) контекстно-свободна ☐

г) регулярна

☐ При кой анализ стекът е дефиниран неявно?

Лексически анализ

Синтактичен анализ отгоре-надолу

Семантичен анализ

Синтактичен анализ отдолу-нагоре ☐

☐ Изберете ВЯРНОТО твърдение:

Всеки регулярен израз може да бъде представен с краен автомат ☐

В семантичния анализ не се използва символна таблица

Бакус-Наур форма (БНФ) е метод за представяне на работата на компилатора

Разделянето на входната програма на tokens се изпълнява от семантичния анализатор

☐ Кой от изброените функции на асемблерна програма са машинно-зависими?

Преместване на генерираната програма в оперативната памет

Формат на инструкциите

Всички изброени ☐

Режим на адресиране

☐ Интерпретаторите предоставят по-добра диагностика за грешки, защото:

Изпълняват входната програма ред по ред ☐

Оптимизират входната програма

Създават междинен байт код

Използват виртуална машина

☐ Ако  $X$  е терминален символ, то  $\text{First}(X)$  е:

Първият нетерминален символ след  $X$

Самият терминален символ ☐

Името на първата синтактична функция

Първият терминален символ след  $X$

☐ Регулярна граматика е:

Нито едно от изброените

Контекстно-свободна граматика, чиито правила съдържат единствен нетерминал от лявата страна и произволен низ от символи от дясната страна ☐

Граматика на английския език

раматика, която не е контекстно-свободна

Контекстно-свободна граматика с ограничения

Контекстно-свободна граматика, чиито правила съдържат празен низ

☐ Коя от следните програми е резидентна в оперативната памет?

Асемблер

Зареждаща програма ☐

Текстов редактор

Свързващ редактор

☐ Кое е ГРЕШНОТО твърдение за синтактичен анализатор отгоре-надолу?

Използва явен (експлицитно дефиниран) стек ☐

Прилага правилата отляво надясно

Построява синтактичното дърво от корена към листата

Избира правило и се опитва да разпознава нетерминални символи в изречението

☐ Дадена е граматика с нетерминали  $N = \{S, C, S1\}$ , терминали  $T = \{a, b, i, t, e\}$ ,  $S$  - стартов символ и следните правила:

$S \rightarrow iCtSS1 \mid a$

$S1 \rightarrow eS \mid \epsilon$

$C \rightarrow b$

е ляво рекурсивна

е дясно рекурсивна

е еднозначна ☐

не е контекстно-свободна

☐ Кое от следните е парсер отгоре-надолу?

Рекурсивно спускане ☐

Анализатор на прецедента на операторите

LR(k) парсер

LALR(k) парсер

☐ Кое от изброените действия описва по подходящ начин (приложимо за LR parsing)?

Това е позицията в изречението, в която ще се извърши следната операция за изместване или намаляване

Това е нетерминал, чиято продукция ще бъде използвана за редукция в следваща стъпка

Това е продукция, която може да бъде използвана за редукция в бъдеща стъпка, заедно с позиция в изреченска форма, където ще се извърши следващата операция за изместване или редукция

Това е продукцията  $p$ , която ще бъде използвана за редуциране в следващата стъпка, заедно с позицията в изреченската форма, където може да се намери дясната страна на продукцията ☐

☐ Граматиката  $S \rightarrow aSa \mid bS \mid c \epsilon$ :

LL(1), но не LR(1)

LR(1), но не LL(1)

и LL(1), и LR(1) ☐

нищо LL(1), нищо LR(1)

☐ Оптимизиращ компилатор:

а) се оптимизира, за да заема по-малко място

б) е оптимизиран, за да отнема по-малко време за изпълнение

**в) оптимизира кода** ☐

г) Нито едно от изброените

☐ Кое от следните се използва за групиране на символите в токени?

а) Парсер

б) Оптимизация на кода

в) Генериране на код

**г) Лексически анализ** ☐

☐ Какъв е изходът на лексическия анализатор?

Набор от регулярни изрази

Синтактични дървета

**Набор от токени** ☐

Низ от символи

☐ Кое от следните не е регулярен израз?

$[(a+b)^*-(aa+bb)]^*$

**$[(0+1)-(0b+a1)^*(a+b)]^*$**  ☐

$(01+11+10)^*$

$(1+2+0)^*(1+2)^*$

☐ Кое от следните е вярно?

$(01)^*0 = 0(10)^*$

$(0+1)^*0(0+1)^*1(0+1) = (0+1)^*01(0+1)^*$

$(0+1)^*01(0+1)^*+1^*0^* = (0+1)^*$

**Всички изброени** ☐

☐ Граматика, която поражда повече от едно синтактично дърво за дадено изречение е:

Нееднозначна

Детерминирана

Контекстно-зависима

Регулярна

## ☐ Контекстно-свободна ☐

Без ограничени

☐ Кое от следните твърдения е ГРЕШНО?

а) Контекстно-свободната граматика се използва за лексически и синтактични правила

## ☐ Проверка на типовете се извършва преди синтактичния анализ ☐

в) Програмите, написано на езици от високо ниво могат да се транслират до различни междинни представяния

г) Аргументите на функция могат да се предават чрез програмния стек

☐ Кое от следните е важно за ефективно преобразуване на инфиксен израз в постфиксен?

## ☐ а) Стек от оператори ☐

б) Стек за операнди

в) Стек за операнди и стек за оператори

г) дърво (parse tree)

☐ Проверка на типовете се извършва по време на:

а) Лексически анализ

б) Синтактичен анализ

## ☐ в) Syntax directed translation ☐

г) Оптимизация на кода

☐ Кой от следните методи е най-мощния parsing метод?

LL(1)

## ☐ Canonical LR ☐

SLR

LALR

☐ Дадена е следната граматика:

$S \rightarrow CC$

$C \rightarrow cC \mid d$

а) LL(1) ☐

б) SLR(1), но не LL(1)

в) LALR(1), но не SLR(1)

г) LR(1), но не LALR(1)

☐ При оценката на отдолу-нагоре на синтактично насочена дефиниция, наследените атрибути могат да:

винаги да бъдат оценявани

да се оценяват само ако дефиницията е L-атрибут ☒

да се оценяват само ако дефиницията има синтезирани атрибути

никога да не се оценяват

☐ Кой от следните правила на граматиката нарушават изискванията за операторна граматика? P, Q, R са нетерминали, a, r, s, t са терминали.

1.  $P \rightarrow QR$

2.  $P \rightarrow QsR$

3.  $P \rightarrow \varepsilon$

4.  $P \rightarrow QtRr$

а) само 1

б) само 1 и 3 ☒

в) само 2 и 3

г) само 3 и 4

☐ Следната граматика е дефинирана чрез следните правила с два оператора \* и +.

$S \rightarrow T * P$

$T \rightarrow U \mid T * U$

$P \rightarrow Q + P \mid Q$

$Q \rightarrow Id$

$U \rightarrow Id$

+ е ляво асоциативен, докато \* е дясно асоциативен

+ е дясно асоциативен, докато \* е ляво асоциативен ☒

И двете + и \* са дясно асоциативни

И двете + и \* са ляво асоциативни

☐ а) И двете P и Q са верни

б) P е вярно, Q е грешно

в) P е грешно, Q е вярно ☒

г) И двете P и Q са грешни

☐ Граматиката  $A \rightarrow AA \mid (A) \mid \varepsilon$  не е подходяща за предсказващ parsing, защото граматиката е:

а) еднозначна ☒

б) ляво рекурсивна

- в) дясно рекурсивна
- г) операторна граматика

Кой от следните класове твърдения обикновено създава неизпълним код, когато се компилира?

декларация ✓

- декларации за присвояване
- входни и изходни оператори
- структурни оператори

В сравнение с програмата, преведена/транслирана от компилатор, същата програма, когато е интерпретирана се изпълнява:

по-бързо

по-бавно ✓

- със същата скорост
- може по-бързо или по-бавно

Граматика, която е ляво и дясно рекурсивна за нетерминал е:

- еднозначна
- нееднозначна

информацията не е достатъчна, за да се реши дали е еднозначна или нееднозначна ✓

нито едно от изброените

Една от целите за използване на междинен код в компилаторите е да:

- направи синтактичния и семантичния анализ по-лесен
- подобри възстановяването на грешки и докладването на грешки

увеличи шансовете за повторно използване на машинно-независимия оптимизатор на кода в други компилатори ✓

подобри разпределението на регистрите

Дадени са следните твърдения:

1. Лексическия анализ се задава чрез контекстно-свободни граматики и се реализира чрез pushdown автомати
2. Синтактичния анализ се задава чрез регулярни изрази и се реализира от машина с крайно състояние (краен автомат)

- само 1
- само 2

И двете 1 и 2

Нито 1, нито 2 ✓

В компилатора ключовите думи на езика се разпознават по време на:

синтактичния анализ на програмата

генерирането на код

лексическия анализ на програмата ✓

анализ на потока от данни

Дадени са следните твърдения:

1. Символните таблици са достъпни само по време на лексически и синтактичен анализ.
2. Компилаторите за програмните езици, които поддържат рекурсия, задължително се нуждаят от хранилище за паметта за разпределяне (allocation) в средата за изпълнение
3. Грешки, които нарушават условието “всяка променлива трябва да бъде декларирана преди използването и“, се откриват по време на синтактичния анализ.

а) само 1

б) само 1 и 3

в) само 2

г) Нито 1, 2 и 3 ✓

Кое от следните е ГРЕШНО?

LALR анализатор изпълнява анализа отдолу-нагоре

Алгоритъм за синтактичен анализ, който извършва сканиране отляво надясно и отклонение най-вдясно е RL(1) ✓

LR анализатор изпълнява анализа отдолу-нагоре

в LL(1), 1 показва, че има поглед само един символ напред

Коя от следните фази на компилатора генерира поток от атоми?

Синтактичен анализ ✓

Лексически анализ

Генериране на код

Оптимизация на код

Кои от следните правила на граматиката нарушават изискванията за операторна граматика? А, В, С са нетерминали, а а, b, c са терминали.

1.  $A \rightarrow BC$
2.  $A \rightarrow CcBb$
3.  $A \rightarrow BaC$
4.  $A \rightarrow \epsilon$

а) само 1

б) само 1 и 2

в) само 1 и 3

г) само 1 и 4 ✓

Кой от следните класове твърдения обикновено създава неизпълним код, когато се компилира?

декларация ✓

декларации за присвояване

входни и изходни оператори

структурни оператори

В сравнение с програмата, преведена/транслирана от компилатор, същата програма, когато е интерпретирана се изпълнява:

по-бързо

по-бавно ✓

със същата скорост

може по-бързо или по-бавно

Граматика, която е ляво и дясно рекурсивна за нетерминал е:

еднозначна

нееднозначна

информацията не е достатъчна, за да се реши дали е еднозначна или нееднозначна ✓

ниито едно от изброените

Една от целите за използване на междинен код в компилаторите е да:

направи синтактичния и семантичния анализ по-лесен

подобри възстановяването на грешки и докладването на грешки

увеличи шансовете за повторно използване на машинно-независимия оптимизатор на кода в други компилатори ✓

подобри разпределението на регистрите

Дадени са следните твърдения:

1. Лексическия анализ се задава чрез контекстно-свободни граматика и се реализира чрез pushdown автомати
2. Синтактичният анализ се задава чрез регулярни изрази и се реализира от машина с крайно състояние (краен автомат)

само 1

само 2

И двете 1 и 2

Нито 1, нито 2 ✓

В компилатора ключовите думи на езика се разпознават по време на:

синтактичния анализ на програмата

генерирането на код

лексическия анализ на програмата ✓

анализ на потока от данни

Дадени са следните твърдения:

1. Символните таблици са достъпни само по време на лексически и синтактичен анализ.
2. Компиляторите за програмните езици, които поддържат рекурсия, задължително се нуждаят от хранилище за паметта за разпределяне (allocation) в средата за изпълнение
3. Грешки, които нарушават условието “всяка променлива трябва да бъде декларирана преди използването и“, се откриват по време на синтактичния анализ.

а) само 1

б) само 1 и 3

в) само 2

г) Нито 1, 2 и 3 ✓

Кое от следните е ГРЕШНО?

LALR анализатор изпълнява анализа отдолу-нагоре

Алгоритъм за синтактичен анализ, който извършва сканиране отляво надясно и отклонение най-вдясно е RL(1) ✓

LR анализатор изпълнява анализа отдолу-нагоре

в LL(1), 1 показва, че има поглед само един символ напред

Коя от следните фази на компилатора генерира поток от атоми?

Синтактичен анализ ✓

Лексически анализ

Генериране на код

Оптимизация на код

Кои от следните правила на граматиката нарушават изискванията за операторна граматика? A, B, C са нетерминали, a, b, c са терминали.

1.  $A \rightarrow BC$

2.  $A \rightarrow CcBb$

3.  $A \rightarrow BaC$

4.  $A \rightarrow \epsilon$

а) само 1

б) само 1 и 2

в) само 1 и 3

г) само 1 и 4 ✓

☐ Кое от следните извеждания прави синтактичния анализатор отдолу-нагоре по време на синтактичния анализ?

ляво извеждане

ляво извеждане наобратно

дясно извеждане

дясно извеждане наобратно ☒

☐ Кое от следните извеждания прави синтактичния анализатор отгоре-надолу по време на синтактичния анализ

на входен низ?

ляво извеждане ☒

ляво извеждане наобратно

дясно извеждане

дясно извеждане наобратно

☐ Предсказващия синтактичен анализатор (парсер) налага някои ограничения на граматиката и приема само клас на граматиката, известен като  $LL(k)$  граматика.

вярно ☒

грешно

може да е и вярно, и грешно

не може да се каже

☐ Вид рекурсивно спускане, което не изисква връщане назад (backtracking) е:

☒ предсказващ ☐

непредсказващ

рекурсивен

нерекурсивен

☐ В кой от следните методи синтактичният анализатор стартира процеса на ново като използва различни правила

за същата продукция, ако едно извеждане на продукцията се провали?

отдолу-нагоре

рекурсивно спускане

☒ връщане назад (backtracking) ☐

всички изброени

☐ Когато синтактичният анализатор (парсера) започне създаването на дървото от стартовия символ и след това

опита да преобразува стартовия символ във входа, входа се нарича:

отдолу-нагоре

☒ отгоре-надолу ☐

и двете а) и б)

нито едно от изброените

☐ На колко типа е разделен синтактичният анализ?

☒ 2 ☐

3

4

5

☐ Изходът от асемблер се нарича:

даннов файл

файл за задачи

☒ обектен файл ☐

програмен файл

☐ Какво прави компилатора, ако настъпи грешка?

терминира програмата

спира изпълнението

чете цялата програма, дори и ако срещне грешки ☒

дава предупреждение

☐ Каква прави интерпретатора, ако настъпи грешка?

терминира програмата

спира изпълнението ☒

чете цялата програма, дори и ако срещне грешки

дава предупреждение

☐ Кое от следните създава вход за компилаторите?

интерпретатор

асемблер

зареждаща програма

препроцесор ☒

☐ Системния софтуер, който транслира сорс кода в обектен код е:

асемблер

компилатор

интерпретатор

езиков процесор ☒

☐ Как се нарича процеса, който се използва за преобразуване на набор от инструкции или програма, които са написани на език от високо ниво, в инструкции (или програма), които могат да се изпълняват на компютър?

асемблиране

компилиране ☒

транслиране

качване

☐ Асемблерния език:

е най-лесния език за писане на програми

използва азбучни кодове вместо двоични числа, използвани в машинен език ☒

не е нужно да бъде преобразуван в машинен език

нито едно от изброените

☐ Кой език се разбира лесно/директно от компютъра?

а) Език от високо ниво

б) Асемблер

в) **Машинен език** ☐

г) Естествен

☐ Компютърна програма, която преобразува асемблерен език в машинен език е

Компилятор

Интерпретатор

**Асемблер** ☐

Компаратор

☐ Компютърна програма, която преобразува една програма в машинен език наведнъж е:

Интерпретатор

**Компилятор** ☐

Симулатор

Конвертор

☐ Как се нарича транслятор, който използва програма написана на език от високо ниво като входна програма и създава машинен код като изход?

Асемблер

**Компилятор** ☐

Интерпретатор

Дебъгер

Коя памет превежда/транслира един ред наведнъж?

**Интерпретатор** ☐

Транслатор

Симулатор

Нито едно от изброените

В контекста на компилаторите, кое от изброените не е междинно представяне на изходната програма?

Три адресен код

Абстрактно синтактично дърво (AST)

Графика на потока на управление (CFG)

**Символна таблица** ☐